МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод Ньютона.

Студент гр. 4342 Закаблук А.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 5. Функция $f(x) = arctg(1/x) - x^2$.

Изучить метод Ньютона для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Теоретический минимум.

В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$, сходящейся к корню уравнения $f(x) = 0$. Достаточные условия сходимости метода формулируются теоремой, приведенной в [1,7].

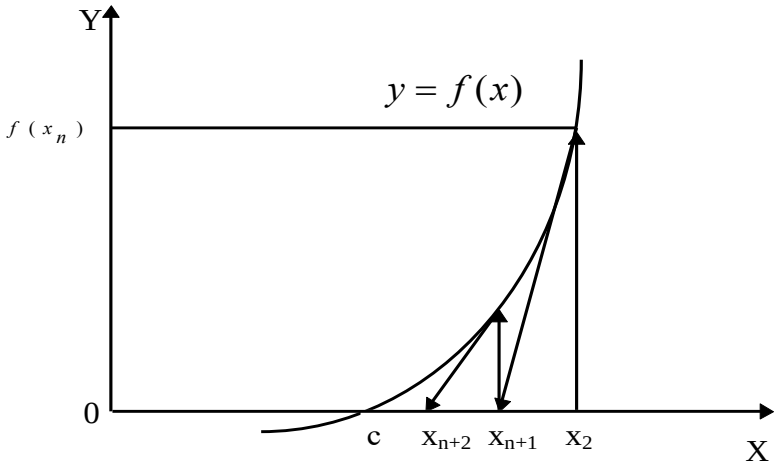

Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью Ox будет очередным приближением x_{n+1} корня уравнения $f(x) = 0$.

Для оценки погрешности n-го приближения корня предлагается пользоваться неравенством

$$|c - x_n| \le \frac{M_2}{2m_1}|x_n - x_{n-1}|^2,$$

где M_2-наибольшее значение модуля второй производной $|f''(x)|$ на отрезке [a,b]; m_1-наименьшее значение модуля первой производной $|f'(x)|$ на отрезке [a,b]. Таким образом, если $|x_n - x_{n-1}| < \varepsilon$, то $|c - x_n| \le \frac{M_2 \varepsilon^2}{2m_1}$. Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место квадратическая сходимость). Из указанного следует, что при необходимости нахождения корня с точностью ε итерационный процесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon_0 = \sqrt{2m_1 \varepsilon / M_2}. \quad (3.1)$$

Рассмотрим один шаг итераций. Если на (n-1)-м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1}), f'(x_{n-1})$ и следующие приближение корня $x_n = x_{n-1} - f(x_{n-1}) / f'(x_{n-1})$. При выполнении условия (3.1) величина x_n принимается за приближенное значение корня с, вычисленное с точностью ε.

Выполнение работы.

1) Графический способ. С помощью графического веб-калькулятора desmos.com построим график (Рисунок 1) для определения отрезка $[a, b]$ близкого к $f(x) = 0$ на котором выполняется условие $f(a) * f(b) < 0$.

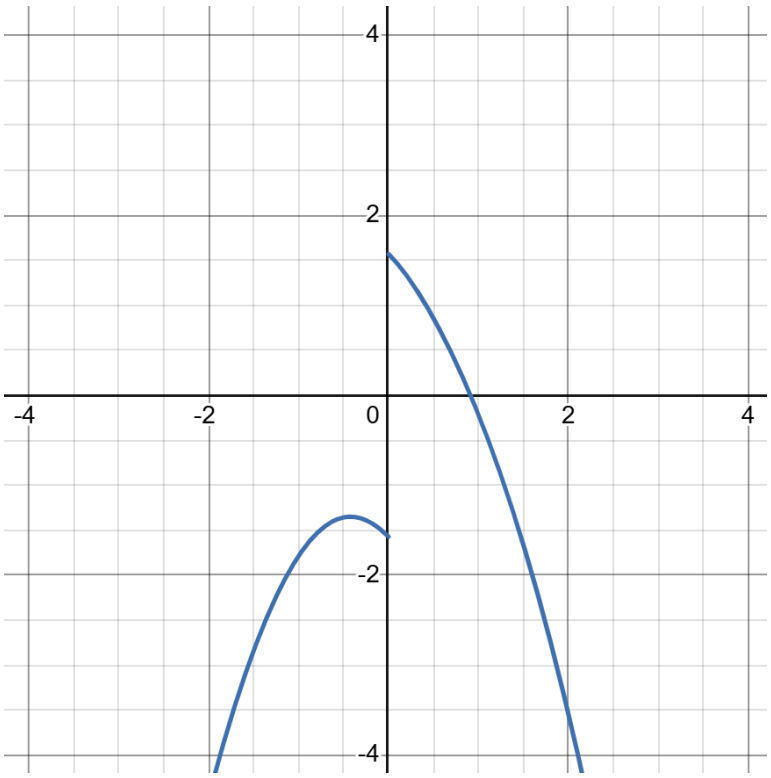

Рисунок 1 - график функции $f(x) = arctg(1/x) - x^2$

На графике видно, что корень находится около значения $x \approx 0.915$. Можно взять отрезок, где $a \in (0.1, 0.91)$, а $b \in (0.92, 2.0)$. Для простоты и надежности возьмем отрезок [0.5,1.5]. Проверка: $f(0.5) \approx 0.857 > 0$, $f(1.5) \approx -1.662 < 0$. Условие выполняется.

Аналитический способ (аналитическое отделение корня):

Для доказательства наличия и единственности корня на выбранном отрезке исследуем функцию аналитически с помощью производной. Найдем первую производную функции: $f`(x) = -1/(x^2+1) - 2x$.

Рассмотрим поведение производной на интервале $(0,+\infty)$. Очевидно, что при любых $x>0$ выражения $(-1/(x^2+1))$ и $(-2x)$ строго меньше нуля. Следовательно, $f'(x) < 0$ при $x \in (0,+\infty)$. Так как производная знакопостоянна и отрицательна, функция f(x) монотонно и строго убывает на всем интервале. Это гарантирует, что функция может пересечь ось Ox не более одного раза (корень единственный).

Проверим выполнение условий теоремы Коши (Больцано-Коши) на границах выбранного отрезка [a,b]=[0.5,1.5]:

- $f(0.5)=arctg(1/0.5)-0.5^2=arctg(2)-0.25\approx1.107-0.25=0.857$
- $f(1.5)=arctg(1/1.5)-1.5^2=arctg(2/3)-2.25\approx0.588-2.25=-1.662$

  Поскольку функция f(x) непрерывна на отрезке [0.5,1.5] и принимает на его концах значения разных знаков (f(0.5)>0, а f(1.5)<0, следовательно f(a)·f(b)<0), то на этом отрезке существует единственный корень. Аналитическое отделение корня успешно выполнено.

  Выбор начального приближения x_0: Согласно условиям сходимости метода Ньютона, начальное приближение x0 нужно выбрать из отрезка [0.5,1.5] так, чтобы выполнялось условие f(x0)·f″(x0)>0 (в методичке также приводится упрощенное следствие f(x)·f′(x)>0). Мы знаем, что первая производная f′(x)<0 и вторая производная ($f''(x)=2x/(x^2+1)^2-2$) f″(x)<0 на всем отрезке. Следовательно, чтобы произведение было больше нуля, сама функция f(x0) тоже должна быть отрицательной. На концах отрезка мы имеем: f(0.5)>0 и f(1.5)<0. Следовательно, в качестве начального приближения необходимо выбрать правый конец отрезка: $x_0=1.5$.

2) Функция `extern double F(double x)` теперь возвращает значение `std::atan(1.0, x) - (x * x)` в соответствии с вариантом $f(x)=arctg(1/x)-x^2$. Исходный код в Приложении А.
3) Создана функция, находящаяся в файле main.cpp в Приложении А.
4) Проведены вычисления, построен график (Рисунок 2).

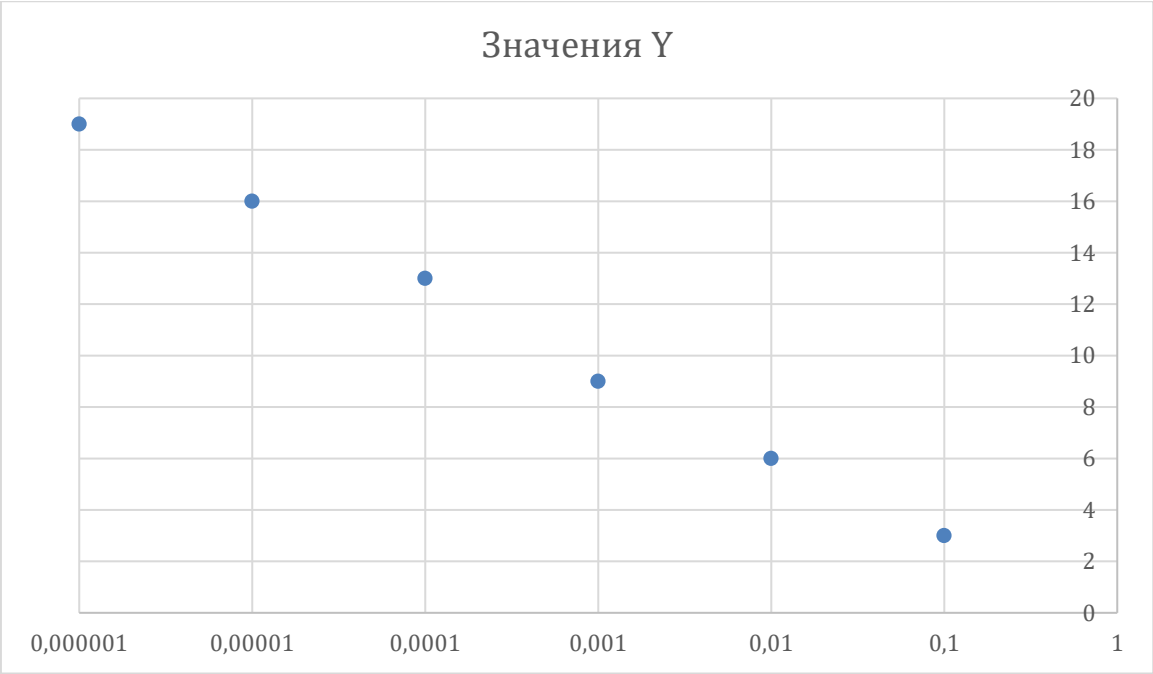

Рисунок 2 - “График” зависимости количества итераций от точности вычислений

Можно заметить, что на Рисунке 2 с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

5) Для исследования чувствительности добавлены флаги компиляции ROUND и DELTA. При ROUND=1 DELTA задается автоматически значением 0.001, если нужно можно задать отдельно. Ниже в Таблице 1 приведены значения при разных дельта.

Таблица 1 - исследование чувствительности метода к ошибкам в исходных данных

Значение DELTA	ε	Значение возвращаемое функцией NEWTON	Количество итераций необходимых для заданной ε.
10^{-1}	10^{-1}	0.904848	2
	10^{-2}	0.904848	2
	10^{-3}	0.904848	2

	10^{-4}	0.904848	2
	10^{-5}	0.904848	2
	10^{-6}	0.904848	2

Продолжение Таблицы 1

Значение DELTA	ε	Значение возвращаемое функцией NEWTON	Количество итераций необходимых для заданной ε.
10^{-3}	10^{-1}	0.913848	2
	10^{-2}	0.911741	3
	10^{-3}	0.911741	3
	10^{-4}	0.911741	3
	10^{-5}	0.911741	3
	10^{-6}	0.911741	3
10^{-5}	10^{-1}	0.914036	2
	10^{-2}	0.911906	3
	10^{-3}	0.911902	4
	10^{-4}	0.911902	4
	10^{-5}	0.911902	4
	10^{-6}	0.911902	4
10^{-7}	10^{-1}	0.914036	2
	10^{-2}	0.911906	3
	10^{-3}	0.911902	4
	10^{-4}	0.911902	4
	10^{-5}	0.911902	4
	10^{-6}	0.911902	5

Выводы.

В ходе работы был изучен метод Ньютона. Для уравнения $f(x) = arctg(1/x) - x^2$ корень был успешно найден на отрезке [0.5,1.5] с точностью от 0.1 до 0.000001. Теоретические данные полностью подтвердились экспериментально: метод Ньютона обладает квадратичной скоростью сходимости. Для достижения высочайшей точности (10^{-6}) потребовалось всего 4 итерации, что значительно быстрее методов бисекции (19 итераций) и хорд.

Однако исследование обусловленности показало, что метод Ньютона крайне чувствителен к ошибкам в исходных данных (особенно при вычислении производной). Округление значений функции и производной с низкой точностью ($\Delta=10^{-1}$) приводит к искажению угла наклона касательной, из-за чего алгоритм останавливается на неточном значении и дальнейшее уменьшение ε не улучшает результат.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <cmath>
#include "methods.hpp"

#ifndef X0_START
    #define X0_START 1.5 // Начальное приближение
#endif

int main() {
    int iterationsCount;
    std::cout << "Eps\t\t | Root\t\t\t | Iterations\n";
    std::cout << "--------------------------------------------------\n";
    for (double epsilon = 0.1; epsilon >= 0.000001; epsilon /= 10) {
        std::cout << epsilon << "\t\t | "
                  << NEWTON(X0_START, epsilon, iterationsCount) << "\t\t | "
                  << iterationsCount << '\n';
    }
    return 0;
}
```

Файл methods.cpp

```
#include "methods.hpp"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#ifndef ROUND_FLAG
    #define ROUND_FLAG 0
#endif
#ifndef DELTA
    #define DELTA 0.001
#endif

#if !ROUND_FLAG
extern double F(double x) {
    // Вариант 5: f(x) = arctg(1/x) - x^2
    return atan(1.0 / x) - (x * x);
}
// Добавляем вычисление первой производной для метода Ньютона
extern double F1(double x) {
    // Производная: f'(x) = -1 / (x^2 + 1) - 2x
    return (-1.0 / (x * x + 1.0)) - (2.0 * x);
}
#else
extern double F(double x) {
    return Round(atan(1.0 / x) - (x * x), DELTA);
}
// Производная с учетом погрешности (округления)
extern double F1(double x) {
    return Round((-1.0 / (x * x + 1.0)) - (2.0 * x), DELTA);
}
#endif

double HORDA(double Left, double Right, double Eps, int &N)
{
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;
```

```
    if (FLeft * FRight > 0.0) {
        puts("Неверное задание интервала\n");
        exit(1);
    }
    if (Eps <= 0.0) {
        puts("Неверное задание точности\n");
        exit(1);
    }

    N = 0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    do
    {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);
        if (Y == 0.0) return (X);

        if (Y * FLeft < 0.0)
        {
            Right = X;
            FRight = Y;
        }
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    while ( fabs(Y) >= Eps );

    return(X);
}

double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;
    if (FLeft * FRight > 0.0) {puts("Неверное задание интервала\n");exit(1);}
    if (Eps <= 0.0) {puts("Неверное задание точности\n");exit(1);}
    N=0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;
    while ((Right - Left) >= E)
    {
        X = 0.5 * (Right + Left);
        Y = F(X);
        if (Y == 0.0) return (X);
        if (Y * FLeft < 0.0) Right = X;
        else { Left = X; FLeft = Y; }
        N++;
    };
    return(X);
}

double Round(double X, double Delta)
{
```

```
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0) return (Delta * (long((X / Delta) + 0.5)));
    else         return (Delta * (long((X / Delta) - 0.5)));
}

double ITER(double X0, double Eps, int &N)
{
    if (Eps <= 0.0) {puts("Неверное задание точности\n"); exit (1);}
    double X1 = F(X0);
    double X2 = F(X1);
    N = 2;
    while( (X1 - X2)*(X1 - X2) > fabs((2*X1 - X0 - X2)*Eps) )
    {
        X0 = X1;
        X1 = X2;
        X2 = F(X1);
        N++;
    }
    return(X2);
}

#ifdef __NEWTON
double NEWTON (double X, double Eps, int &N)
{
    extern double F1 (double);
    double Y, Y1, DX;
    N = 0;
    do
    {
        Y = F(X);
        if (Y == 0.0) return (X);

        Y1 = F1(X);
        if (Y1 == 0.0) {
            puts("Производная обратилась в ноль\n");
            exit(1);
        }

        DX = Y / Y1;
        X = X - DX;
        N++;
    }
    while (fabs(DX) > Eps);

    return (X);
}
#endif
```

Файл methods.hpp

```
#ifndef METHODS_HPP
#define METHODS_HPP

#define __NEWTON

extern double F(double);

#ifdef __NEWTON
extern double F1(double);
#endif

double Round(double X, double Delta);
```

```
double BISECT(double Left, double Right, double Eps, int &N);
double ITER(double X0, double Eps, int &N);
double HORDA(double Left, double Right, double Eps, int &N);
double NEWTON(double X, double Eps, int &N);

#endif // METHODS_HPP
```